AI23531 – DEEP LEARNING

EX 6 BIDIRECTIONAL RNN VS FEED-FORWARD NETWORK FOR TIME-SERIES PREDICTION

AIM

To implement a bidirectional recurrent neural network (BiRNN) to predict sequences in time-series data, train it, and compare its performance with a traditional feed-forward neural network (Airline Passenger dataset).

PROCEDURE

1. Load the monthly airline passenger data and scale it to 0–1 with MinMaxScaler.
2. Create input sequences of 10 past values with the next value as the target.
3. Split the sequences into 80% training and 20% test sets and create PyTorch DataLoaders.
4. Define a BiRNN (bidirectional RNN with 64 hidden units + linear layer) and a feed-forward network (10 → 64 → 1).
5. Train both with Adam (lr = 0.01) and MSE loss for 100 epochs.
6. Predict on the test data, inverse-scale the predictions and plot them against the actual values.
7. Compare the models using Mean Squared Error (MSE).

ALGORITHM

1. Start.
2. Read the dataset and normalise it.
3. Generate sliding-window sequences (length 10) and targets; split into train and test sets.
4. Build the BiRNN and the feed-forward model.
5. For each model, for 100 epochs: forward pass, compute MSE loss, backpropagate, update weights with Adam.
6. Predict the test sequences, convert back to the original scale.
7. Plot actual vs predicted values and compute the MSE of each model.
8. Stop.

PROGRAM

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler


url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/refs/heads/master/monthly-
airline-passengers.csv'
df = pd.read_csv(url, usecols=[1])
data = df.values.astype('float32')
scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(data)


SEQ_LENGTH = 10
X = np.array([data_scaled[i:i+SEQ_LENGTH] for i in range(len(data_scaled) -
SEQ_LENGTH)])
y = np.array([data_scaled[i + SEQ_LENGTH] for i in range(len(data_scaled) -
SEQ_LENGTH)])
```

```
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

import torch
from torch.utils.data import TensorDataset, DataLoader

X_train_tensor = torch.tensor(X_train)
y_train_tensor = torch.tensor(y_train)
X_test_tensor = torch.tensor(X_test)
y_test_tensor = torch.tensor(y_test)
train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=16, shuffle=True)
test_loader = DataLoader(TensorDataset(X_test_tensor, y_test_tensor), batch_size=1)

import torch.nn as nn

class BiRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=1):
        super(BiRNN, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True,
bidirectional=True)
        self.fc = nn.Linear(hidden_size * 2, 1)
    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])

class FeedforwardNN(nn.Module):
    def __init__(self, input_size):
        super(FeedforwardNN, self).__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 1)
    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.fc2(self.relu(self.fc1(x)))

def train_model(model, loader, epochs=100):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    model.train()
    for epoch in range(epochs):
        loss_epoch = 0
        for seqs, targets in loader:
            optimizer.zero_grad()
            outputs = model(seqs)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()
            loss_epoch += loss.item()
        if (epoch + 1) % 20 == 0:
            print(f"Epoch {epoch+1}/{epochs}, Loss: {loss_epoch / len(loader):.5f}")

def evaluate_model(model, X):
    model.eval()
    with torch.no_grad():
        return model(X).numpy()

birnn = BiRNN()
train_model(birnn, train_loader)
ffnn = FeedforwardNN(input_size=SEQ_LENGTH)
train_model(ffnn, train_loader)
pred_birnn = evaluate_model(birnn, X_test_tensor)
pred_ffnn = evaluate_model(ffnn, X_test_tensor)
```

```
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

pred_birnn_inv = scaler.inverse_transform(pred_birnn)
pred_ffnn_inv = scaler.inverse_transform(pred_ffnn)
y_test_inv = scaler.inverse_transform(y_test)

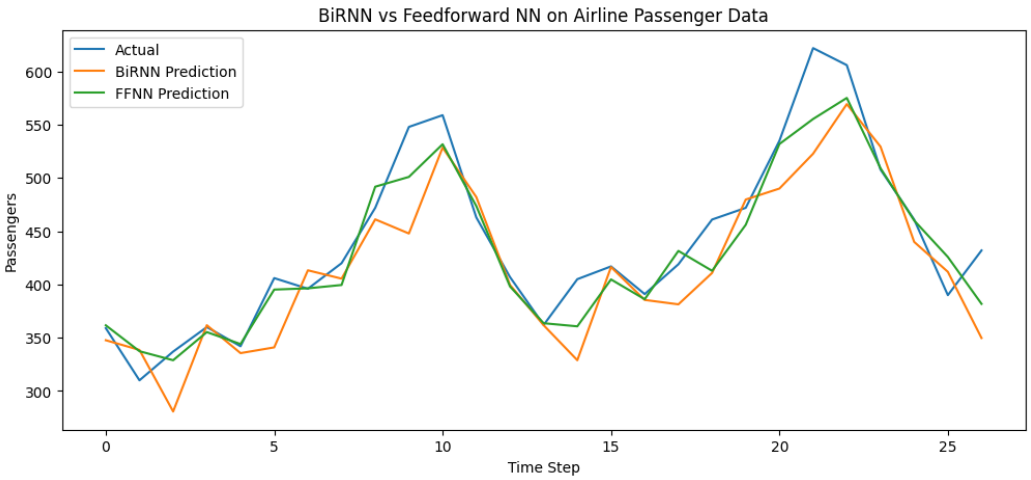
plt.figure(figsize=(12, 5))
plt.plot(y_test_inv, label='Actual')
plt.plot(pred_birnn_inv, label='BiRNN Prediction')
plt.plot(pred_ffnn_inv, label='FFNN Prediction')
plt.legend()
plt.title('BiRNN vs Feedforward NN on Airline Passenger Data')
plt.xlabel('Time Step'); plt.ylabel('Passengers')
plt.show()

mse_birnn = mean_squared_error(y_test_inv, pred_birnn_inv)
mse_ffnn = mean_squared_error(y_test_inv, pred_ffnn_inv)
print(f"BiRNN MSE: {mse_birnn:.3f}")
print(f"FFNN MSE: {mse_ffnn:.3f}")
```

OUTPUT

```
Epoch 20/100, Loss: 0.00263
Epoch 40/100, Loss: 0.00204
Epoch 60/100, Loss: 0.00354
Epoch 80/100, Loss: 0.00245
Epoch 100/100, Loss: 0.00235
Epoch 20/100, Loss: 0.00226
Epoch 40/100, Loss: 0.00231
Epoch 60/100, Loss: 0.00166
Epoch 80/100, Loss: 0.00101
Epoch 100/100, Loss: 0.00079
BiRNN MSE: 1909.219
FFNN MSE: 702.857
```

Console output



BiRNN vs Feed-forward NN on the Airline Passenger data

RESULT

In this run the feed-forward network gave a lower test error (MSE 702.857) than the BiRNN (MSE 1909.219), so the BiRNN did not outperform it on this small dataset.